

AI-1(Auto categorization and pattern recognization)

DATA WE WILL BE GETTING

THROUGH SMS PARSING , WE WILL BE GETTING MSG RECIEVED BY USER FOR EACH AND EVERY TRANSACTIONS . THAT DATA WILL BE STORED TO A DATABASE AND WILL BE USED FOR THE PROCESSING OF AI.

WE WILL ALSO BE GETTING THE LIVE LOCATION OF THE USER WHEN A USER DOES A PAYMENT . LIKE FOR EG THEATRES , RESTAUREANTS, SHOPPING MALLS ETC WHICH CAN BE USED FOR CATEGORIZATION.

Onboarding Questions in the app like

"What best describes you?"

→ [Student] [Working Professional] [Freelancer] [Business Owner]

"Where do you mostly shop?"

→ [Online mostly] [Local shops mostly] [Mix of both]

"Do you cook at home or order food more often?"

→ [Cook at home] [Order food] [Both equally]

WHAT THE AI SHOULD DO

THE AI SHOULD CATEGORIZE THE TRANSACTION ACCORDING TO THE BELOW RULES MENTIONED

IT SHOULD ALSO SAVE AND STORE THE AMOUNT A USER SPENT BY EACH PAYMENT METHOD LIKE UPI, CARD, CASH ETC.

IF UNABLE TO CATEGORIZE ANY TRANSACTION , IT SHOULD BE GIVEN FOR MANUAL REVIEW TO THE USER IN THE REVIEW SECTION OF THE APP

IF FOUND AND RECURRING EXPENSES (NETFLIX SUBSCRIPTION , ELECTRICITY BILL , WIFI BILL), IT SHOULD STORE THE DETAILS IN A SEPERATE DATABASE AND

THE AMOUNT OF THOSE AND SHOULD BE DISPLAYED ON THE RECURRING EXPENSE SECTION OF THE APP.

BETA - THERE WILL BE A DIFFERENT AI THAT WILL BE HAVING THE FULL ACCESS TO READ, WRITE AND SAVE ALL THE DATA AND CATEGORIZATION MADE BY THIS AI . SO EVERYTHING SHOULD CHANGE ACCORDINGLY AS THE DECISION OF THAT AI

Understands the category by keyword (Swiggy , zomato , uber , blinkit , Pharmeasy etc)

Sub-Type	Examples	VPA Signals	Category
Food Delivery	Swiggy, Zomato	swiggy@icici , zomato@hdfc	Food & Dining
Ride Hailing	Uber, Ola, Rapido	uber@icici , ola@okaxis	Transport
E-commerce	Amazon, Flipkart, Meesho	amazon@apl , flipkart@yesbank	Shopping
Grocery	BigBasket, Blinkit, Zepto	bigbasket@icici , blinkit@hdfcbank	Groceries
Streaming	Netflix, Hotstar, Spotify	netflix@icici , hotstar@ybl	Entertainment
Travel	IRCTC, MakeMyTrip, RedBus	irctc@sbi , mmtravelspay@icici	Travel
Pharmacy	1mg, PharmEasy, Netmeds	1mg@icici , pharmeasy@hdfc	Health
Fuel	HPCL, BPCL, Indian Oil	hpcl@hpcl , bpcl@sbi	Fuel

SOLU: The Ai should extract keywords from the message or the Recieving merchant UPI Id and should categorize accordingly

2. Transferring to a person directly

Real SMS examples:

textINR 500.00 debited from A/c XX4821 on 12-Apr to VPA 9876543210@ybl. Ref No 4123456789.

INR 15000.00 paid to Rahul Sharma via UPI.
UPI Ref: 40123456789

SOLU : After each direct transaction happens and the AI identifies that it cannot be directly classifies , a pop up notification should come which have a quick drop down menu which ca categorize as (Food, Travel, Entertainment, Others). If the user didnt select any of these options from the pop up notification or clears the notification , this transaction should be shown in the review part of the app from where user can add this to any category once the user opens the app.

3. Payment to a merchant

Paying directly to a QR code of an offline merchant

Paid ₹240 to SHARMA GENERAL STORE via UPI

VPA: sharmag@paytm

Paid ₹80 to 9988776655@upi - Auto driver

Paid ₹1200 to RAVI MEDICALS via UPI

VPA: ravimedical@oksbi

Sub-Type	VPA Clues	Amount Signal	Category
Kirana / Local Grocery	"store", "mart", "general", "provision" in name	₹50–₹2,000	Groceries
Dhaba / Local Restaurant	"hotel", "restaurant", "dhaba", "kitchen", "biryani"	₹50–₹800	Food & Dining
Auto / Cab Driver	Bare mobile number VPA, no brand	₹30–₹500	Transport
Medical / Pharmacy	"medical", "pharma", "clinic", "hospital"	₹50–₹5,000	Health
Salon / Barber	"salon", "beauty", "hair", "spa"	₹50–₹2,000	Personal Care
Tailor / Repairs	"tailors", "works", "service", "repair"	₹100–₹5,000	Services
Vegetable/Fruit Vendor	Very small amounts + mobile VPA	₹20–₹500	Groceries

SOLU: The ai can use the combined concept of above table and also the location details to classify correctly the transactions .

IMP : THE USER HAS TO FULL ACCESS TO CHANGE THE CATEGORY WHICH THE AI AUTOMATICALLY CATEGORIZED AT ANY POINT OF TIME INSIDE THE APP . SO IF A USER DOES THAT , THE AI SHOULD REMEMBER THAT VENDOR AND SHOULD REMEMBER THAT ANY TRANSACTION DONE BY THE USER TO THE SAME VENDOR WILL BE THE CATEGORY THAT WAS CHANGED TO BY THE USER.

EG: USER DID A TRANSACTION TO A MERCHANT NAME S.S WATER SUPPLY AND THE UPI ID WAS XXXXXXXXX. SO THE AI CATEGORIZES IT INTO HOUSEHOLD EXPENSES. LATER IN THE APP WHILE REVIEWING THE PAST TRANSACTIONS , HE FOUND THAT THE TRANSACTION AI CATEGORIZED WAS TO HOUSEHOLD TRANSACTIONS , BUT THE USER HAD ACTUALLY PAID THAT FOR LAUNDRY SERVICES . SO THE USER CHANGED THAT TO SERVICES. SO AI SHOULD NOTE DOWN THAT AND IN THE FUTURE IF THE USER DOES A TRANSACTION TO THIS MERCHANT (CROSS VERIFYING THE USERS LOCATION, THE AI SHOULD CATEGORIZE IT INTO SERVICES ONLY.

4. Bill Payment (Recurring expenses)

These are the transaction which mostly happens every month.

Real examples:

textINR 847.00 debited from A/c XX1234 to BESCOM ELECTRICITY
Ref: BBPS/20240412/001234

ECS DEBIT: INR 499.00 - AIRTEL BROADBAND
Narration: AIRTEL_FIBER_APR

INR 2399.00 Auto-debit TATA SKY ANNUAL

Sub-Type	Keywords / Billers	Amount Pattern	Category	
Electricity	BESCOM, MSEDCL,	₹300–₹5,000, monthly	Utilities	

Sub-Type	Keywords / Billers	Amount Pattern	Category	
	BSES, TNEB, CESC			
Mobile Recharge	Jio, Airtel, Vi, BSNL	₹149–₹999, 28/84-day cycle	Telecom	
DTH	Tata Sky, Dish TV, Airtel DTH, Sun Direct	₹200–₹1,000, monthly	Entertainment	
Gas Bill	MGL, IGL, Adani Gas, Mahanagar Gas	₹400–₹2,000, monthly	Utilities	
Water Bill	Municipal corporation	₹100–₹500, quarterly	Utilities	
Internet/Broadband	ACT Fibernet, Hathway, Jio Fiber	₹500–₹1,500, monthly	Telecom	
Insurance Premium	LIC, HDFC Life, SBI Life	Monthly/annual, varies	Insurance	

SOLU: The AI should extract the merchant name through keywords in the SMS. **Key training signal:** BBPS transactions almost always contain the word "BBPS", "ECS DEBIT", or "NACH" in the SMS narration — use these as strong categorical flags, not just merchant name.

IMP: WHEN THE AI FINDS OUT SUCH A TRANSACTION, THE AI SHOULD VERIFY WHETHER THIS TYPE OF TRANSACTION IS HAPPENING EVERY SINGLE MONTH . IF THIS HAPPENS , THEN THE AI SHOULD STORE THE RECURRING EXPENSES IN A SEPERATE DATABASE SO THAT WE CAN SHOW THE USERS WHAT ALL RECURRING EXPENSES ARE THERE FOR HIM AND ALSO AROUND HOW MUCH.

5. ATM & Card Transactions (Non-UPI)

These come through debit/credit card usage — both ATM cash withdrawals and physical card swipes

Real examples:

textINR 5000.00 withdrawn from ATM at HDFC Bank ATM
BANGALORE on 12-Apr-26

INR 1349.00 debited from A/c XX4821 for POS TXN at
LIFESTYLE STORES - FORUM MALL on 11-Apr-26

INR 12000.00 debited - POS at SAMSUNG EXPERIENCE STORE

Sub-Type	SMS Keywords	Amount Signal	Category
ATM Cash Withdrawal	"ATM", "withdrawn", "cash withdrawal"	₹500–₹20,000	Cash Withdrawal
POS - Retail Shopping	"POS", "debit card", mall/store name	₹200–₹50,000	Shopping
POS - Grocery	"POS" + supermarket name (D-Mart, More)	₹200–₹5,000	Groceries
POS - Fuel Station	"POS" + petrol/fuel/HP/BP/IOC	₹300–₹5,000	Fuel
POS - Restaurant	"POS" + restaurant name	₹200–₹10,000	Food & Dining

SOLU : POS transactions give you the **merchant name in full text** (e.g., "LIFESTYLE STORES - FORUM MALL") — use character n-gram NLP on this field for classification, not just keyword matching.

6. EMI, Loan , Investments

These are high-importance transactions for financial behavior tracking. They are almost always auto-debits with very clear narration patterns.

Real examples:

textINR 8542.00 debited - EMI towards loan A/c XX7834
HDFC BANK HOME LOAN EMI

INR 15000.00 debited - NACH DEBIT - BAJAJ FINANCE
Narration: BAJAJ_EMI_PERSONAL_LOAN

INR 5000.00 debited to HDFC CREDIT CARD PAYMENT

Sub-Type	Keywords	Category
Home Loan EMI	"HOME LOAN EMI", "housing loan"	Loan EMI
Personal Loan EMI	"PERSONAL LOAN", "BAJAJ", "IIFL"	Loan EMI
Car / Vehicle Loan	"AUTO LOAN", "VEHICLE EMI"	Loan EMI
Credit Card Payment	"CREDIT CARD PAYMENT", "CC PMT"	Credit Card
Mutual Fund SIP	"SIP", "GROWW", "ZERODHA", "COIN@ZERODHA"	Investment
Stock Investment	"ZERODHA", "UPSTOX", "GROWW", debit	Investment
Fixed Deposit	"FD BOOKING", "FIXED DEPOSIT"	Investment

SOLU : Should categorize according to the keywords and should be stored in load, credit card or investment category.

7. Salary and Income

Credits (money coming in) must be identified separately — they are not expenses and should not be categorized into spending buckets. They should be categorized into Income.

INR 45000.00 credited to A/c XX4821 - NEFT from
ACME SOLUTIONS PVT LTD Ref: SALARY APR 2026

INR 500.00 credited - UPI from 9876543210@ybl

Remarks: freelance payment

INR 83.00 credited - INTEREST CREDIT Q4

Sub-Type	Keywords	Category
Salary	"SALARY", employer name credit, NEFT large credit	Income - Salary
Freelance	Small-medium credits from individuals	Income - Freelance
Cashback / Refund	"CASHBACK", "REFUND", "REVERSAL"	Refund
Interest	"INTEREST CREDIT", "INT CREDIT"	Income - Interest
Family / Gift	Credits from family VPAs	Personal Transfer

IMP: if a large amount of money get credited into my account , the AI should as the user whether this is salary . If the user confirms , then remember that this is the salary(range) he will be getting. If similar range of amount gets credited to users account at the same date range of every single month , then auto categoruze it to income every single month . REMEMBER APPROVAL OF USER AS THIS IS SALARY IS NECESSARY FOR ONE TIME.

8. Other cases

Wallet top-up	"INR 2000 transferred to Paytm Wallet"	It's not a spend yet	Tag as "Wallet Load", not an expense
----------------------	--	----------------------	--------------------------------------

Reversed/Failed txn	"REVERSAL of TXN 40123456789"	Must not count as spend	Detect "REVERSAL", "FAILED", "REFUND" → exclude
----------------------------	-------------------------------	-------------------------	---

If refund happens , it should not be counted as expense as well as income . If a particular amount of money was debited to some merchant and the same amount of money was refunded after a period of time by the same merchant, then it should not be calculated as income as well as expense.

IMPORTANT CASE SCENARIOS

Pooling

Pooling happens when a group of people spends for something and one user pays and the other user pays there own split back to the user.

eg. A group of friends went for some food. They were a group of five people, so I paid for the whole food. I paid 2500 and they paid 500 by four other people after

two days. So I received 2000 back so that 2000 should not be counted as my income and the whole 2500 should not be counted as the. Charges also it should be added as 500 only.

The Transaction Linking Engine

This is a separate layer that sits **after** your categorization model and operates on the transaction graph. It has 4 internal components.

Component 1: Reimbursement Candidate Detector

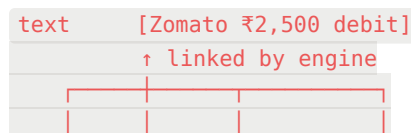
Every incoming P2P credit is flagged as a **potential reimbursement** if it meets these criteria:

Signal	Condition	Weight
Source type	Credit comes from an individual VPA (mobile number or name), not a brand/BBPS	High
Time proximity	Credit arrives within 0–7 days of an outgoing debit	High
Amount is round/divisible	₹500, ₹250, ₹333 — clean split amounts	Medium
Amount < debit amount	The credit is smaller than the linked debit	High
Multiple credits, same window	More than one person pays around the same time	Very High
Recurring contact	This VPA has sent money before after user's outings	Medium

If a credit scores high on these signals, it's added to a **Reimbursement Candidate Pool** with a pending link, not classified as income.

Component 2: Transaction Graph Builder

You model all transactions as a **directed graph** in memory:



```
[Rahul [Priya [Ankit [Meena
₹500] ₹500] ₹500] ₹500]
```

```
→ SplitGroup created:
Anchor: Zomato ₹2,500
Members: 4 reimbursements
Total reimbursed: ₹2,000
Net user expense: ₹500
```

The engine builds this graph by running the **Amount Correlation Algorithm** below.

Component 3: Amount Correlation Algorithm

This is the math that proves whether a set of credits is linked to a debit. It checks three conditions:

Rule 1 — Divisibility Check

Does the debit amount divide evenly into N equal parts (for N = 2 to 10)?

```
text₹2,500 ÷ 5 = ₹500 ✓ (exact match with incoming credits)
₹2,500 ÷ 4 = ₹625 ✗
₹2,500 ÷ 3 = ₹833 ✗
```

Match found at N=5. This is a very strong signal.

Rule 2 — Sum Convergence Check

Do the credits received (within the time window) sum to approximately $(N-1)/N \times$ debit?

```
textCredits received: ₹500 + ₹500 + ₹500 + ₹500 = ₹2,000
Expected reimbursement: ₹2,500 × (4/5) = ₹2,000 ✓
```

Rule 3 — Time Window Validation

Are all credits within an acceptable window after the debit?

```
textDebit: April 10
Credits: April 11, April 12, April 12, April 13 ✓ (all within 7 days)
```

If 2 out of 3 rules pass → flag as **high confidence split**. If 1 rule passes → flag as **possible split** and ask user to confirm.

Component 4: The 4 Real-World Split Scenarios Your Engine Must Handle

Scenario A — Post-Paid Split (your example)

User pays full → friends reimburse later

```
textDay 1: Debit ₹2,500 → Zomato [Mark as "Pending reconciliation"]
Day 2: Credit ₹500 ← Rahul [Candidate detected, linked to Day 1]
Day 3: Credit ₹500 ← Priya, Ankit, Meena [All linked]
Result: Net expense = ₹500, Food & Dining
₹2,000 tagged as "Reimbursement Received" (not income)
```

Scenario B — Pre-Paid Collect

User collects money from friends first, then pays full

```
textDay 1: Credit ₹500 ← Priya [Suspicious – not from brand, small amount]
Day 1: Credit ₹500 ← Rahul, Ankit, Meena [Multiple small credits in hours]
Day 1: Debit ₹2,500 → Swiggy [Engine looks back, finds 4 credits = ₹2,000]
Result: Net expense = ₹500
₹2,000 tagged as "Collected for group"
```

Scenario C — User Pays Only Their Share (someone else organized)

User receives a collect request and pays their portion

```
textDay 1: Debit ₹500 → 9876543210@ybl (Rahul)
Context: No large debit from user → user is reimbursing someone else
Result: This IS the user's real expense = ₹500, Food & Dining
Categorize by time (lunch hour) + amount (meal-sized)
```

Scenario D — Partial Reimbursement (someone never pays back)

User gets ₹1,500 back instead of ₹2,000

```
textDebit: ₹2,500 → Zomato
Credits received: ₹1,500 (3 friends paid, 1 didn't)
Net expense = ₹2,500 - ₹1,500 = ₹1,000
```

```
After 7-day window expires with incomplete reimbursement:
→ Lock the net as ₹1,000 expense
→ Optionally flag: "Meena hasn't paid back ₹500 yet"
```

Data Structure Design

You need a new entity in your database alongside regular transactions:

```
textSplitGroup {
  id: UUID
  anchor_transaction_id: string // The original big debit
  anchor_amount: float // ₹2,500
  anchor_merchant: string // Zomato
  anchor_category: string // Food & Dining
}
```

```

member_transactions: [           // The reimbursement credits
  { txn_id, vpa, amount, timestamp, status: "received" }
]

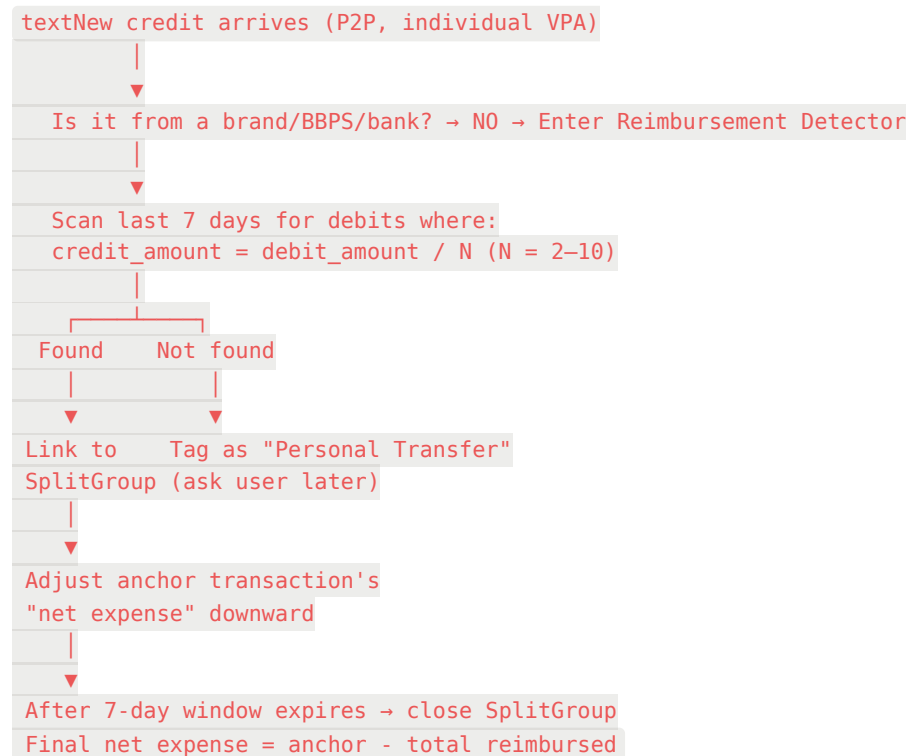
total_reimbursed: float           // ₹2,000
net_user_expense: float           // ₹500 (anchor - reimbursed)

group_size_detected: int          // 5
reimbursement_window_days: int    // 7

confidence_score: float           // 0.0 to 1.0
status: "pending" | "confirmed" | "partial" | "closed"
user_confirmed: boolean
}

```

The Decision Flow



Messy Case	ML Solution
Unequal splits (Rahul paid ₹600, rest paid ₹467)	Train a regression model to predict "is this credit a reimbursement?" based on amount proximity, time, contact history

Messy Case	ML Solution
Same friend always pays you back for different reasons	Build a per-contact behavioral model — learn that Rahul's credits are always reimbursements, never income
User gets ₹500 from Rahul both as reimbursement AND separately as a birthday gift	Context classifier — time of day, day of week, note text, proximity to a debit
Group of 3 but amounts are ₹833, ₹834, ₹833 (rounding)	Fuzzy sum matching with $\pm 5\%$ tolerance on the sum convergence check

Train this with user confirmation data — every time a user says "yes, this is a reimbursement" or "no, this is separate income", that becomes a labeled training sample for your model.

UX Flow That Makes This Work

The key insight is: **you can't fully automate this silently**. The first time, show the user what you detected and let them confirm:

text💡 App noticed something

You paid ₹2,500 to Zomato on Apr 10.
You received ₹500 each from Rahul, Priya, Ankit, and Meena over the next 2 days.

Looks like a group expense! 🗳️

Your actual share: ₹500

[Yes, split this] [No, count separately]

Once confirmed once, Dekho remembers:

- This social group (Rahul + Priya + Ankit + Meena) often splits with you
- Future splits from this group are auto-confirmed with high confidence
- This becomes your **social spending graph** — which directly powers App's "Spending Patterns" and "Emotional Spending" features in your Behavior tab

Special Edge Case: Wallet-Mediated Splits

Some users split via Splitwise/settle-up apps and then do one final UPI payment. In this case you only see one P2P credit (Rahul pays ₹2,000 as a single sum). Your engine should still handle this:

```
textDebit ₹2,500 → Zomato  
Credit ₹2,000 ← Rahul (single settlement)  
Net = ₹500 ✓
```

```
Rule: Any single credit that is exactly (N-1)/N  
of a recent debit, from a known social contact,  
triggers a split suggestion
```

Spendings made for Family

In the transaction details , there will be options by default to change the category of the transaction by user it self . But if the user long presses the transaction , an option will come to add that to family expenses . User can do this to categorize expenses like Child Education Fees , Parent hospital cases etc.

Asking review from the user for every direct transaction made and recieved

If any transaction is recieved directly from a person or sent directly to a person , a popup will come like as for what this expense was made. If a money was recieved from a person directly , a small question will be asked like was this a gift , reimbursement for split or income.

Cashback & Rewards — Silent Dictionary Tagger

The solution: A two-layer detection system that never bothers the user.

Layer 1 — Known Source Dictionary

Maintain a static list of all cashback/reward senders. These are identifiable by their VPA or SMS sender ID:

```
textCASHBACK_SOURCES = {
  VPA patterns: ["hdfcbankltd@", "icicibank@", "sbicard@",
                "phonepe.rewards@", "amazonpay.refund@",
                "paytmcashback@", "gpay.cashback@"],

  SMS keywords: ["CASHBACK", "CASH BACK", "REWARD CREDITED",
                "LOYALTY POINTS", "OFFER CREDIT", "PROMO CREDIT",
                "REFERRAL BONUS", "SCRATCH CARD"],

  Amount range: ₹1 – ₹2,000 (cashbacks are rarely larger)
}
```

Layer 2 — Origin Linking

When a cashback credit is detected, link it backward to the original spend:

```
text₹999 debit → BigBasket (Apr 10)      → Groceries
₹100 credit ← HDFC Cashback (Apr 11)    → Linked to BigBasket txn
```

Display in app:

BigBasket	₹999
└ Cashback	-₹100
Net spend:	₹899

Store both gross and net values — show **gross spend** in transaction history, **net spend** in monthly summary insights. This way the user sees both their real cost and their savings.

What to surface in insights:

```
text"You saved ₹430 in cashbacks this month 💰
Highest cashback: HDFC on BigBasket (₹100)"
```

1. Wallet Top-Up — Float Tracking System

The solution: Treat wallets as a **virtual sub-account** inside Dekho, not as a spend.

Step 1 — Wallet Load Detection

```
textWALLET_MERCHANTS = {
  "Paytm Wallet": VPAs containing "paytm", SMS: "Paytm wallet loaded"
  "PhonePe Wallet": VPAs containing "ybl" + "wallet" keyword
  "Amazon Pay": VPAs containing "amazonpay" + "wallet"
  "Mobikwik": "mobikwik@"
}
```

When detected → **don't count as expense** → move ₹2,000 to "Wallet Float" sub-account.

Step 2 — Downstream SMS Capture

Some wallets DO send SMS per spend. Capture and categorize these normally, but deduct from wallet float instead of bank balance:

```
textWallet Float: ₹2,000 loaded
└ ₹80 Ola Auto (Transport) [from wallet]
└ ₹120 Kirana (Groceries) [from wallet]
└ ₹340 Swiggy (Food) [from wallet]
Remaining Float: ₹1,460
```

Step 3 — The Hard Case (No Downstream SMS)

When wallet spends don't generate SMS (common with Paytm wallet), the float just disappears silently. Handle this with a **monthly reconciliation prompt** — not transaction by transaction, just once:

```
text💡 "You loaded ₹2,000 into Paytm Wallet last month.
We couldn't track how it was spent."
```

Move this section to the review section where user will enter money or percentage manually and will categorize accordingly.

Step 4 — Learn Over Time

After 2–3 months of user responses, build a **per-user wallet spending profile**:

```
textJobin's Paytm wallet usage pattern:
60% Transport (auto/cab), 30% Food, 10% Misc
→ Auto-assign future untracked wallet floats using this profile
→ Stop asking after confidence threshold is reached
```

Booking Refunds from Merchants — Merchant Refund Detector

The solution: A separate refund detection pipeline that operates only on **merchant credits** (not P2P credits).

Step 1 — Separate Merchant Credits from P2P Credits

Every incoming credit needs to be classified first:

```
textCredit received
|
```

- └ Source is a known brand VPA → MERCHANT CREDIT pipeline
- └ Source is a person VPA/number → P2P pipeline (you already handle this)

Step 2 — Merchant Credit Sub-Classification

textMERCHANT CREDIT TYPES:

1. Refund/Cancellation

Signal: Credit from merchant within 1–30 days of a debit to same merchant

Signal: SMS contains "REFUND", "REVERSAL", "CANCELLED", "RETURNED"

Action: Reverse the original debit amount (partial or full)

Crosschecking the refund source

2. Cashback

Signal: Credit from bank (not merchant) referencing a merchant

Signal: Amount is small (< 20% of any recent debit)

Action: Tag as cashback, link to original spend

3. Seller Payment (for resellers/freelancers)

Signal: Credit from Meesho, Amazon Seller, Flipkart Seller VPA

Action: Tag as "Business Income" – not personal income

4. Insurance / Claim Credit

Signal: Credit from LIC, insurance VPA

Action: Tag as "Insurance Claim" – not income

Step 3 — Refund Linking Logic

textCredit ₹2,000 ← MakeMyTrip (Apr 15)

Look back 90 days for:

→ Debit to MakeMyTrip between ₹1,800 – ₹50,000

Found: Debit ₹25,000 → MakeMyTrip (Jan 10)

Action:

- Link refund to original booking
- Reduce original transaction net value by ₹2,000
- Tag credit as "Partial Refund - Travel Booking"
- Never count as income

Step 4 — The Deposit Trap

Security deposits to individual landlord VPAs are debits that are NOT expenses:

textDetection: Large P2P debit + "deposit"/"advance"/"security" in any app-added note
OR recurring VPA that receives ₹X and later sends back ~₹X

Action: Tag as "Recoverable Deposit"

→ Show in Assets section of Dekho (not expense)

→ Alert when money comes back: "Your ₹8,000 deposit was returned ✓"

IMP: AT ANY POINT OF TIME THE REFUND IS NOT ABLE TO BE UNDERSTOOD , THE TRANSACTION CAN BE GIVEN FOR USER TO VERIFY . DO NOT MAKE ANY UNCONFIRMED CATEGORIZATION TO MAINTAIN THE ACCURACY OF THE AI

Festival Calendar — Static Dataset + Contextual Framing

The solution: A hardcoded Indian festival calendar that modifies the **tone and threshold** of your insights engine — not the categorization itself.

Step 1 — The Calendar Dataset

```
pythonINDIAN_FESTIVAL_WINDOWS = [
    # (festival, typical_month, window_days_before, window_days_after)
    ("Diwali", "Oct-Nov", 10, 5),
    ("Holi", "Mar", 5, 3),
    ("Eid ul-Fitr", "Mar-Apr", 7, 3),
    ("Eid ul-Adha", "Jun-Jul", 7, 3),
    ("Navratri", "Oct", 9, 2),
    ("Durga Puja", "Oct", 5, 3),
    ("Christmas", "Dec", 7, 2),
    ("Onam", "Aug-Sep", 5, 3),
    ("Pongal/Sankranti", "Jan", 3, 3),
    ("Ugadi/Gudi Padwa", "Mar-Apr", 3, 2),
    ("Raksha Bandhan", "Aug", 5, 2),
    ("Wedding Season", "Nov-Feb", 0, 90), # entire window
    ("Back to School", "Jun-Jul", 0, 30) # education spike
]
```

Note: Since festival dates shift yearly by the Hindu/Islamic calendar, maintain the **actual dates per year** as an updatable config file — not hardcoded months.

Step 2 — Three-Layer Response System

Situation	Normal Month	Festival Window
Spend is 0–20% above average	No alert	No alert
Spend is 20–50% above average	Mild insight	Celebratory framing: <i>"Diwali spending is up — totally normal 🧨"</i>
Spend is 50–100% above average	Overspend warning	Soft framing: <i>"Big festive month! Here's where it went"</i>

Situation	Normal Month	Festival Window
Spend is 100%+ above average	Strong alert	Still flag, but gently: <i>"Even for Diwali, this is quite high. Want to review?"</i>

Step 3 — Post-Festival Recovery Insight

After the festival window ends, surface a recovery nudge:

```
text"Diwali's done! You spent ₹38,000 vs your usual ₹18,000.
That's ₹20,000 extra. Your next 2 months will naturally
feel tighter — here's how to balance it out. 💡"
```

This turns a potential user trust-breaking moment into a **signature Dekho insight** — exactly the emotional safety and clarity your product promises.

Mixed Basket Merchants

Instead of guessing with ratios, you flag suspicious transactions into a **Review Queue** — a dedicated section where the user manually splits or confirms what they spent. One-time action per transaction, maximum accuracy.

Step 1 — Detection: What Gets Flagged for Review

Your algorithm needs to identify transactions that are genuinely ambiguous. Flag a transaction if it matches **2 or more** of these signals:

Signal 1 — Mixed Basket Merchant Dictionary

```
textMIXED_BASKET_MERCHANTS = {
  Online:
    ["Amazon", "Flipkart", "Myntra", "Meesho", "JioMart",
     "Nykaa", "Tata CLiQ", "Ajio", "Snapdeal", "FirstCry"]
  Physical Retail:
    ["DMart", "Big Bazaar", "Reliance Fresh", "More Supermarket",
     "Spencer's", "Hypercity", "Star Bazaar", "Spar", "Lulu Mall"]
  Mall / Complex POS:
    ["FORUM MALL", "PHOENIX", "SELECT CITYWALK", "LULU MALL",
     "INORBIT", "NEXUS", "MALL OF INDIA", "EXPRESS AVENUE",
     "MANTRI SQUARE", "ORION MALL", "VR MALL", "DLFD MALL"]
  General POS pattern:
    Any POS transaction where merchant name contains:
```

```
"MALL", "HYPERMARKET", "SUPERSTORE", "WHOLESALE",
"SUPERMARKET", "RETAIL", "MART", "BAZAAR"
}
```

Signal 2 — Amount Threshold

Only flag if amount is above a minimum threshold — small amounts are almost always single-category:

```
text₹0 - ₹300 → Auto-categorize, never flag (single item purchase)
₹300 - ₹800 → Flag only if merchant is in mixed basket list
₹800 - above → Always flag if merchant is in mixed basket list
```

Signal 3 — No Strong Category Signal

If the merchant name contains a strong category keyword (e.g., "PHARMACY", "MEDICAL", "PETROL"), don't flag even if it's a large amount — it's already categorizable.

Step 2 — The Review Queue UI Flow

When flagged, the transaction doesn't sit uncategorized — it goes into a visible **"Needs Review"** card in the Expenses tab.

The Review Card (passive, not a popup):

```
text
| 🛒 DMart - ₹2,400 Apr 12 |
| Looks like a mixed shopping trip |
| Help us split this for better insights |
| [Review & Split →] |
```

It doesn't interrupt the user — it sits in a "To Review" section at the top of Expenses and disappears once actioned.

Inside the Review & Split screen:

```
textDMart ₹2,400 - Apr 12, 3:45 PM
```

Split this into what you bought:

[+ Add item]

Groceries	₹1,400	[×]
Personal Care	₹600	[×]
Clothing	₹400	[×]
Total:	₹2,400	✓

[Save Split]

The user adds line items with amounts. The remaining unassigned amount is shown live so they know when it balances to zero. Once saved → those category splits flow directly into the monthly summary.

Step 3 — Smart Defaults Inside the Review Screen

To reduce the effort of manually typing every item, pre-populate the review screen with **educated guesses** as starting points (not final categories):

textYou're at DMart. Common purchases here:

Quick add:

[🥬 Groceries] [🧴 Personal Care] [🧹 Household] [👕 Clothing]

Tap to add → then just adjust the amounts

User taps "Groceries" → a row appears with ₹2,400 pre-filled. They change it to ₹1,400, tap "Personal Care", change to ₹600, and so on. Much faster than typing from scratch.

Step 4 — What If the User Never Reviews?

Not everyone will action the review queue. You need a graceful fallback:

Time elapsed	Action
0–3 days	Sits in "Needs Review" queue, flagged visibly
3–7 days	Gentle nudge: <i>"3 transactions waiting for review"</i>
7 days	Auto-assign to the merchant's primary category (e.g., DMart → Groceries, Amazon → Shopping) with a "Unreviewed" badge
After 30 days	Lock it as primary category, remove badge

The "Unreviewed" badge is key — it signals to the user that this number might be approximate, without breaking their monthly summary.

Shared Subscription

If a large amount was deducted for subscription like 1200 for netflix, a notification and review will go for the user like was this a group subscription or personal subscription and also will be marked for review. The user can decide whether it was single or if group the amount of people. So if the split amount of money was received from any user within a specific amount of days, it will be added as reimbursement

Things to Consider

Confidence Scoring — The Model Must Know What It Doesn't Know

Your categorization model should never output just a category. It must output a **category + confidence score** for every prediction.

```
textInput: "ravimedical@oksbi ₹480"
```

Output:

```
Category: Health & Medical
```

```
Confidence: 0.91 ← high confidence, act silently
```

```
Input: "rahul.kumar@ybl ₹1,200"
```

Output:

```
Category: Personal Transfer
```

```
Confidence: 0.54 ← low confidence, ask user
```

Confidence thresholds:

Score	Action	User Experience
-------	--------	-----------------

Score	Action	User Experience
0.85 – 1.0	Auto-categorize silently	Seamless
0.65 – 0.84	Auto-categorize + show edit option	Subtle
0.45 – 0.64	Show top 2 suggestions, ask user to pick	One tap
Below 0.45	Ask user to categorize manually	Explicit

This is critical because a model that auto-categorizes everything incorrectly with false confidence will destroy user trust permanently. A model that asks when

uncertain feels intelligent and honest

Continuous Learning & Feedback Loop Architecture

Your model must get smarter with every user interaction. Every correction is gold.

Three types of feedback signals:

Explicit feedback (strongest signal)

User manually changes a category → this is a direct labeled training sample

```
textModel said: Shopping
User changed to: Electronics
→ Store: (merchant VPA, amount, time, channel) → Electronics
→ Weight: 1.0 (highest)
```

Implicit feedback (medium signal)

User does NOT correct a categorization after seeing it → soft confirmation that it was right

```
textUser opened transaction, viewed it, didn't edit
→ Store as: weak positive label for that category
→ Weight: 0.3
```

Review completion (strong signal)

User completes a Review Queue split → extremely rich training data

```
textDMart ₹2,400 → [Groceries ₹1,400, Personal Care ₹600, Clothing ₹400]
→ Multi-label training sample for DMart transactions
→ Weight: 0.9
```

Critical architectural decision — where does retraining happen?

Since you're dealing with financial SMS, **on-device training is the right call** for privacy:

```
textDevice collects corrections locally
→ Local model fine-tunes every 7 days on device
→ Only anonymized model weight updates (not raw data) sent to server
→ Server aggregates updates from all users (Federated Learning)
→ Improved global model pushed back to devices
```

This means user data never leaves the phone — crucial for trust in a fintech app in India.

Explainability — Why Did It Categorize This?

For a financial app especially, users will sometimes question the model. You need to be able to explain decisions — both to the user and internally for debugging.

For the user:

```
textUser: "Why is this categorized as Transport?"
```

```
Dekho: "This payment went to 9876543210@upi  
at 8:45 AM on a Monday – similar to  
your usual morning commute pattern 🚗"
```